



Instituto Tecnológico
Superior de Xalapa

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

ING. EN SISTEMAS COMPUTACIONALES

3° Semestre | Grupo: B

ESTRUCTURA DE DATOS

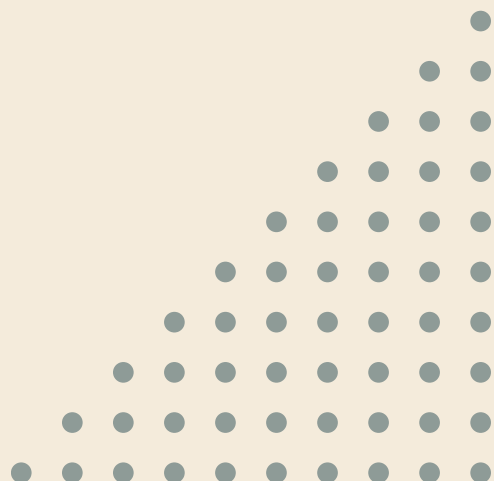
VAZQUES HERRERA VICTOR HUGO

UNIDAD 3 TAREA 1

PRESENTA

257O000081

Brayan Mejia Mora



11 DE OCTUBRE DEL 2026

Pilas Estáticas y Dinámicas en Java

1. Introducción Teórica

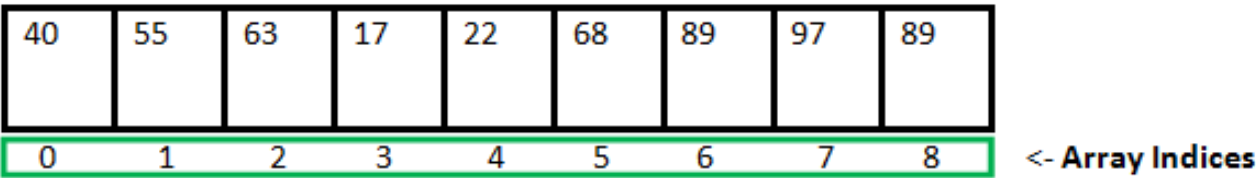
1.1. Concepto de Pila

Una pila (stack) es una estructura de datos lineal que sigue el principio LIFO (Last In, First Out), lo que significa que el último elemento en entrar es el primero en salir. Es análoga a una pila de platos: solo se puede tomar el que está arriba, y solo se puede colocar uno nuevo encima de los demás.

Las operaciones fundamentales de una pila son:

Operación	Descripción
push(x)	Inserta un elemento x en la cima de la pila.
pop()	Elimina el elemento que está en la cima.
peek()	Consulta el elemento de la cima sin eliminarlo.
isEmpty()	Indica si la pila está vacía.
isFull()	Indica si la pila alcanzó su capacidad máxima (solo en la estática).
show()	Recorre y muestra los elementos de la pila.

1.2. Almacenamiento Estático (Contiguo en Memoria)



La pila estática se implementa sobre un arreglo de tamaño fijo. Todos los elementos se almacenan en posiciones contiguas de memoria y se accede a ellos mediante un índice llamado tope. Cuando tope = -1, la pila está vacía.

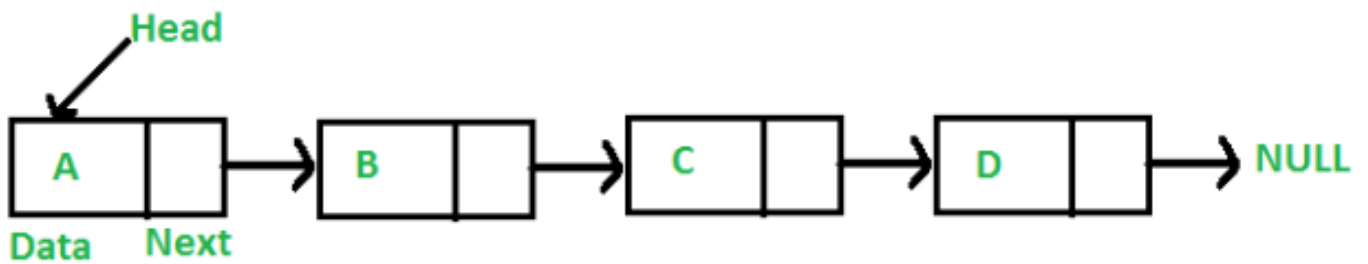
Ventajas:

- Acceso directo y muy rápido a cualquier posición ($O(1)$).
- Uso de memoria predecible y sencillo.
- Menor sobrecarga (overhead) por elemento, ya que no se guardan referencias.

Desventajas:

- Tamaño fijo: si se llena, ocurre overflow (desbordamiento).
- Si se dimensiona muy grande, se desperdicia memoria.

1.3. Almacenamiento Dinámico (Nodos Enlazados)



La pila dinámica se implementa con nodos dispersos en memoria (no contiguos). Cada nodo contiene un dato y una referencia (siguiente) al nodo que está debajo. Solo se mantiene una referencia llamada cima, que apunta al nodo superior.

Ventajas:

- Crece dinámicamente mientras el sistema tenga memoria disponible.
- Uso eficiente de memoria: se reserva nodo por nodo conforme se necesita.
- No existe overflow práctico.

Desventajas:

- Mayor consumo por nodo (dato + referencia).
- No permite acceso aleatorio: solo recorrido secuencial.
- Mayor fragmentación de memoria y sobrecarga del garbage collector.

1.4. Diferencias Conceptuales

Característica	Pila Estática	Pila Dinámica
Base de almacenamiento	Arreglo contiguo	Nodos enlazados
Capacidad	Fija	Ilimitada (según memoria)
Referencia de control	int tope	Nodo cima
Overflow	Sí	No aplica
Underflow	Sí (tope == -1)	Sí (cima == null)
Acceso aleatorio	Sí (datos[i])	No (solo secuencial)
Complejidad push/pop	O(1)	O(1)
Memoria por elemento	4 bytes (int)	4 bytes (int) + referencia

2. Representación Gráfica del Comportamiento LIFO

2.1. Secuencia de operaciones

Tomemos como ejemplo la secuencia: push(10), push(20), push(30), pop(), pop().

Paso 1 — Pila vacía:

Dinámico: cima = null
Estático: (tope = -1) []

Paso 2 — Después de push(10):

Dinámico: cima → [10 | null]
Estático: (tope = 0) [10]

Paso 3 — Después de push(20):

Dinámico: cima → [20 | •] → [10 | null]
Estático: (tope = 1) [10|20]

Paso 4 — Después de push(30):

Dinámico: cima $\rightarrow$ [30 | •] $\rightarrow$ [20 | •] $\rightarrow$ [10 | null]
Estático: (tope = 2) [10|20|30]

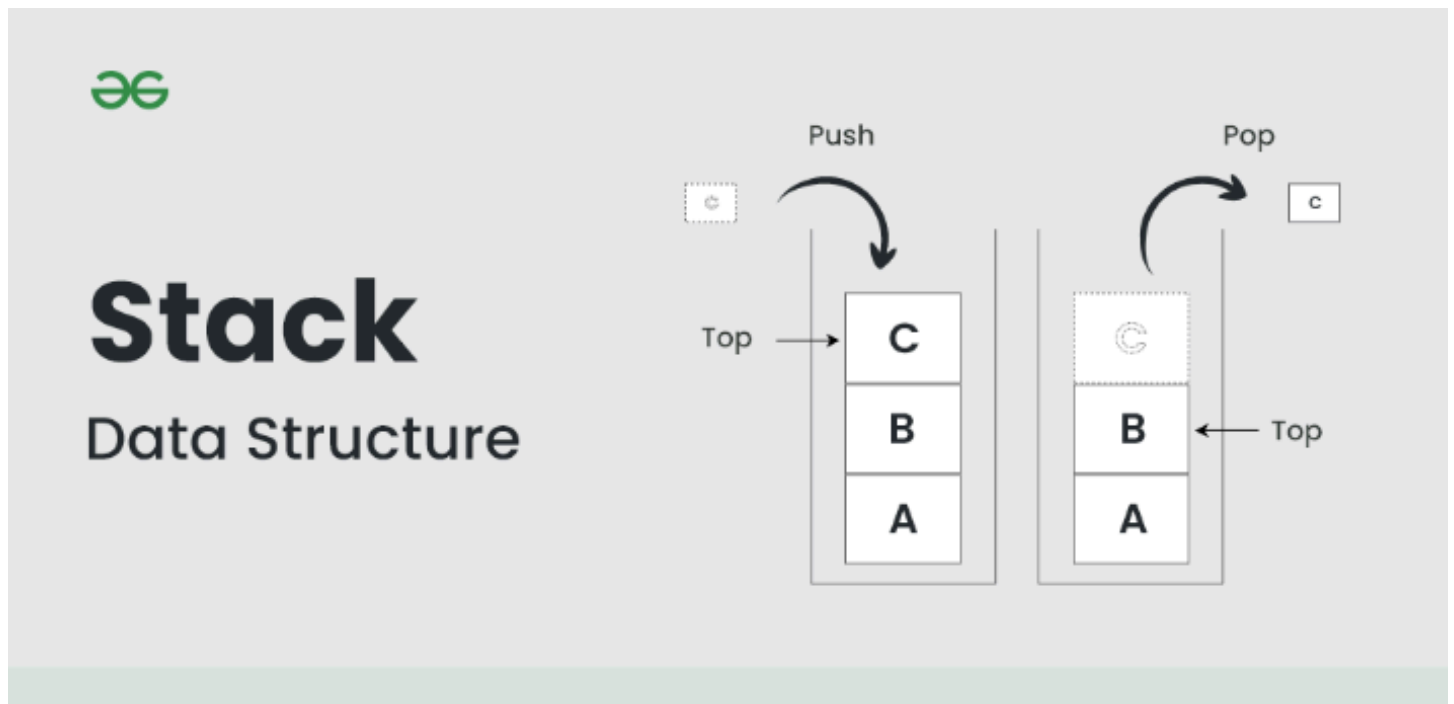
Paso 5 — Después de pop() (sale el 30):

Dinámico: cima $\rightarrow$ [20 | •] $\rightarrow$ [10 | null]
Estático: (tope = 1) [10|20]

Paso 6 — Después de pop() (sale el 20):

Dinámico: cima $\rightarrow$ [10 | null]
Estático: (tope = 0) [10]

2.2. Representación visual del principio LIFO



3. Fragmentos Clave de Código Comentados

3.1. Pila Estática — Método push

```
1  /**
2   * Inserta un elemento en la cima de la pila (operación push).
3   * Antes de insertar verifica que no se haya alcanzado la capacidad máxima.
4   *
5   * @param x el valor entero a insertar
6   */
7  public void push(int x) {
8      if (tope < datos.length - 1) { // ¿Hay espacio disponible?
9          tope++;                     // Avanzamos la cima
10         datos[tope] = x;            // Guardamos el valor
11         System.out.println("Metiste: " + x);
12     } else {
13         System.out.println("Pila llena! No se pudo insertar: " + x);
14     }
15 }
```

Análisis: El método es $O(1)$ porque solo realiza operaciones aritméticas y una asignación. La validación `tope < datos.length - 1` previene el overflow.

3.2. Pila Estática — Método pop

```
1  /**
2   * Elimina el elemento que está en la cima de la pila (operación pop).
3   * Antes de eliminar verifica que la pila no esté vacía.
4   */
5  public void pop() {
6      if (tope >= 0) { // ¿Hay elementos?
7          System.out.println("Sacaste: " + datos[tope]);
8          tope--;      // Retrocedemos la cima
9      } else {
10         System.out.println("Pila vacía! No se puede sacar.");
11     }
12 }
```

Análisis: También es $O(1)$. El dato no se borra físicamente del arreglo; solo se "ignora" al retroceder `tope`. Esto es importante: la memoria sigue ocupada hasta que se sobrescriba.

3.3. Pila Dinámica — Clase Nodo

```
1  /**
2   * Representa un nodo individual de la pila dinámica.
3   * Cada nodo almacena un dato entero y una referencia ("flecha")
4   * al nodo que está debajo de él en la pila.
5   *
6   * Estructura visual:
7   *
8   *     [dato | siguiente] ---> [dato | siguiente] ---> null
9   *
10  */
11  public class Nodo {
12
13      /** Valor entero que guarda este nodo. */
14      public int dato;
15
16      /**
17       * Referencia al siguiente nodo (el que está debajo en la pila).
18       * Si vale null, significa que este es el último nodo (el fondo).
19       * Debe ser del mismo tipo Nodo porque un nodo solo puede apuntar a otro nodo.
20       */
21      public Nodo siguiente;
22
23      /**
24       * Constructor del nodo.
25       * Se inicializa con un valor y con la referencia en null,
26       * para evitar dejar datos "basura" en memoria.
27       *
28       * @param x el valor entero que guardará este nodo
29       */
30      public Nodo(int x) {
31          this.dato = x;           // Nace con un valor concreto
32          this.siguiente = null;  // Al nacer todavía no apunta a ningún nodo
33      }
34  }
```

Análisis: Cada nodo ocupa en memoria más que un simple int: se debe sumar el espacio de la referencia siguiente (típicamente 8 bytes en JVM de 64 bits) más la cabecera del objeto (~16 bytes). Esto es la principal desventaja en memoria de la pila dinámica.

3.4. Pila Dinámica — Método push

```
1  /**
2   * Inserta un nuevo elemento en la cima de la pila (operación push).
3   *
4   * Pasos:
5   *   1. Crear un nuevo nodo con el valor recibido.
6   *   2. El nuevo nodo apunta al que era la cima anterior.
7   *   3. La cima ahora es el nuevo nodo.
8   *
9   * @param x el valor entero a insertar
10  */
11  public void push(int x) {
12      // 1. Creamos un nuevo nodo con el valor x
13      Nodo nuevo = new Nodo(x);
14
15      // 2. El nuevo nodo "atrapa" al que era la cima anterior
16      nuevo.siguiente = cima;
17
18      // 3. El nuevo nodo se convierte oficialmente en la nueva cima
19      cima = nuevo;
20  }
```

Análisis: Operación $O(1)$. La clave está en el orden de las instrucciones: primero se engancha el nuevo nodo al antiguo tope, y después se actualiza cima. Si se hiciera al revés, se perdería la referencia a la pila.

3.5. Pila Dinámica — Método pop

```
1  /**
2   * Elimina el elemento que está en la cima (operación pop).
3   * Antes verifica que la pila no esté vacía.
4   */
5  public void pop() {
6      if (cima != null) {
7          System.out.println("Sacaste de la pila: " + cima.dato);
8          cima = cima.siguiente; // La cima baja al siguiente nodo
9      } else {
10         System.out.println("Pila dinamica vacia! No se puede sacar.");
11     }
12 }
```

Análisis: $O(1)$. Al reasignar `cima = cima.siguiente`, el nodo anterior queda sin referencias y el Garbage Collector de Java lo elimina automáticamente. Aquí sí se libera memoria real.

3.6. Recorrido con nodo auxiliar (Pila Dinámica)

```
1  /**
2   * Muestra todos los elementos de la pila desde la cima hasta el fondo.
3   * No modifica la pila, solo la recorre con un "explorador" temporal.
4   */
5  public void show() {
6      System.out.print("Pila Dinamica (Cima -> Fondo): ");
7
8      if (isEmpty()) {
9          System.out.println("[VACIA]");
10         return;
11     }
12
13     // Recorrido con un nodo auxiliar "actual":
14     // - actual arranca en la cima
15     // - mientras actual != null seguimos bajando
16     // - actual = actual.siguiente salta al nodo de abajo
17     for (Nodo actual = cima; actual != null; actual = actual.siguiente) {
18         System.out.print(actual.dato + " ");
19     }
20
21     System.out.println(); // Salto de línea
22 }
```

Análisis: Se usa un nodo auxiliar actual para no perder la referencia a cima. Si modificáramos cima directamente en el recorrido, destruiríamos la pila.

4. Conclusiones Personales

4.1. Sobre el rendimiento

Ambas estructuras ofrecen complejidad $O(1)$ en push, pop y peek, lo cual las hace igualmente eficientes en cuanto a tiempo de ejecución teórico. Sin embargo, en la práctica existen diferencias:

- La pila estática es ligeramente más rápida porque accede directamente al arreglo, sin seguir referencias. Además, tiene mejor localidad de caché (los datos están juntos en memoria), lo cual es una ventaja real en lenguajes de bajo nivel como C o C++.
- La pila dinámica incurre en costos adicionales: cada push implica una reserva de memoria (`new Nodo(...)`) y cada pop deja basura que el Garbage Collector debe limpiar. En programas con millones de operaciones, esto puede notarse.

4.2. Sobre el uso de memoria

Aquí la diferencia es contundente:

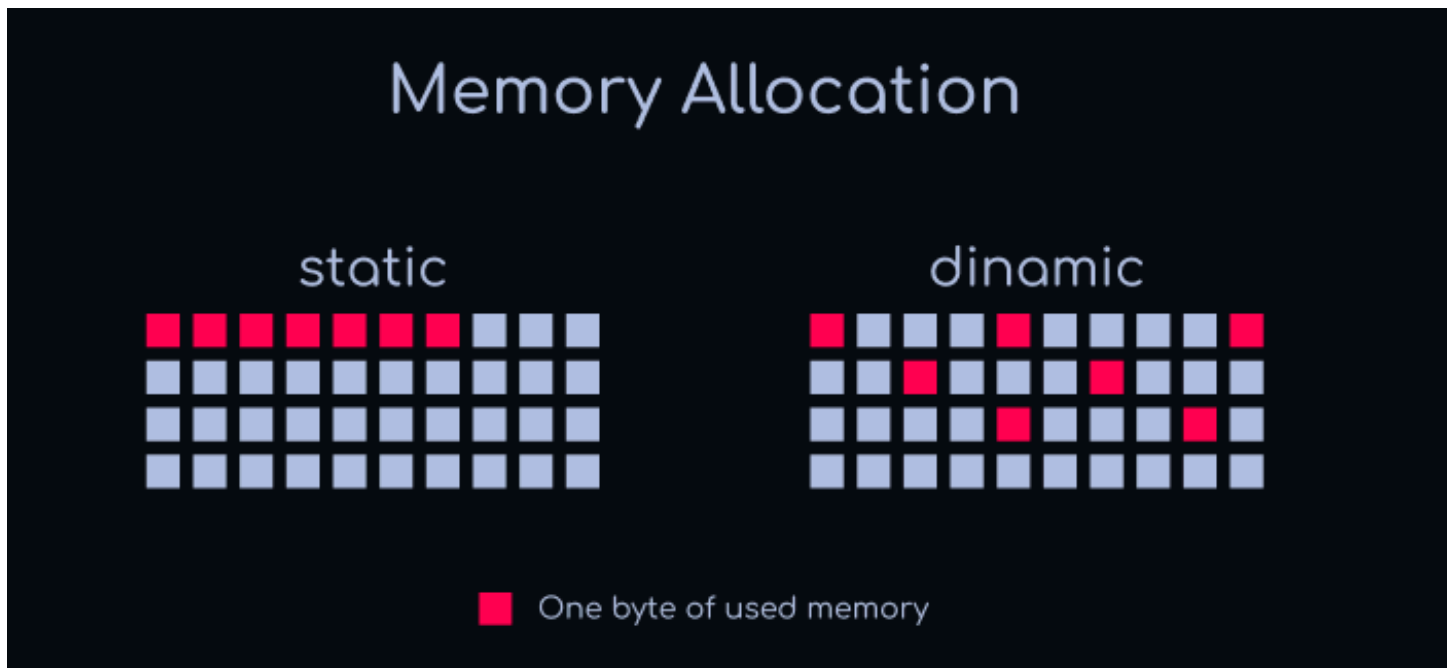
- Un int en un arreglo ocupa 4 bytes.
- Un nodo con un int y una referencia ocupa aproximadamente 24-32 bytes en la JVM (cabecera del objeto + int + referencia + padding).
- Es decir, la pila dinámica puede consumir 6 a 8 veces más memoria que la estática por cada elemento almacenado.

Sin embargo, la pila dinámica no desperdicia memoria cuando no se llena: solo reserva lo que necesita. La estática, en cambio, reserva todo el arreglo desde el inicio, aunque esté vacío.

4.3. Sobre el uso práctico

- Si se conoce de antemano el tamaño máximo y se busca eficiencia y simplicidad, la pila estática es la mejor opción.
- Si el tamaño es variable o desconocido, o si se necesita que la estructura crezca indefinidamente, la pila dinámica es la elección correcta, sacrificando algo de memoria y velocidad.
- En Java, el uso de `ArrayList` o `ArrayDeque` suele ser la opción más equilibrada en producción, ya que combinan arreglos redimensionables con buen rendimiento.

4.4 Reflexión final



Implementar ambas versiones me permitió comprender que no existe una estructura "mejor" en absoluto, sino que cada una responde a un contexto específico. El principio LIFO es el mismo en ambas, pero la forma de materializarlo en memoria cambia radicalmente el comportamiento del programa. Elegir correctamente entre estático y dinámico es una decisión de diseño tan importante como el algoritmo mismo.

Bibliografía

IgKnight: Ignite your tech career - coding courses & interview prep platform. (s/f-a).

Theigknight.com. Recuperado el 6 de octubre de 2026, de

<https://theigknight.com/documentation/datastructures/ds-stack-array-implementation>

IgKnight: Ignite your tech career - coding courses & interview prep platform. (s/f-b).

Theigknight.com. Recuperado el 6 de octubre de 2026, de

<https://theigknight.com/documentation/datastructures/ds-stack-linked-list-implementation>

Stack interview questions. (2025, febrero 26). GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/commonly-asked-data-structure-interview-questions-on-stack/>

Static and dynamic data structures. (2020, abril 30). GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/static-and-dynamic-data-structures-in-java-with-examples/>

Static data structure vs dynamic data structure. (2018, marzo 13). GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/static-data-structure-vs-dynamic-data-structure/>